

CSC 640 Software Quality

The Story

Write down Your Answers

There are no right or wrong answers yet:

- the answers that people think are true (superficial)
- the answers that are actually true (effective)

So, it doesn't matter; write down your (superficial or effective) answers and compare them at the end!

1. What is Software Engineering?
2. What is High-Quality Software?
3. What do we do to build High-Quality Software?
4. What are software process & design?
5. What made developers use high-level languages, not low-level ones?
6. What made developers use automated tools, not manual ones?
7. What made developers use high-level frameworks, not low-level libraries?

We can be more specific for 5 - 7:

5. What made Microsoft create TypeScript when JavaScript seemed good enough?
6. Why do most front-end developers use React instead of manual DOM manipulation?
7. Why do many PHP backend developers use the Laravel **framework** instead of raw PHP?

The Story in One Line

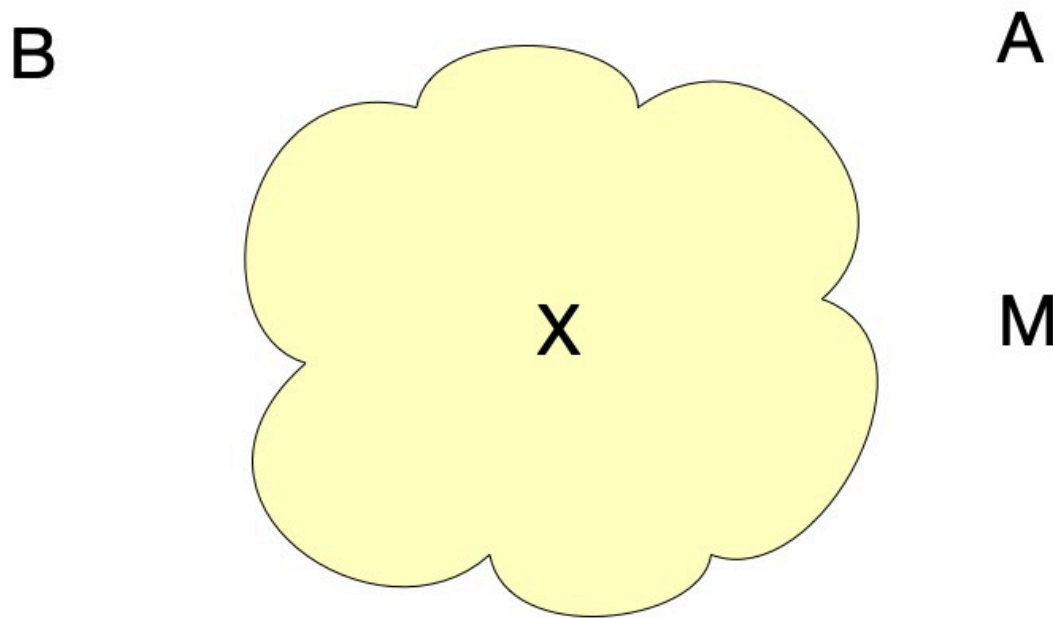
Software engineering is (largely) **managing complexity under constraints.**

1. **Complexity** is the enemy.
2. **Constraints** (money, time, people) make it harder.
3. **Learning early** (prototypes, iterations) exposes what we don't know.
4. **Process**, **design**, and **automation** help us manage it.

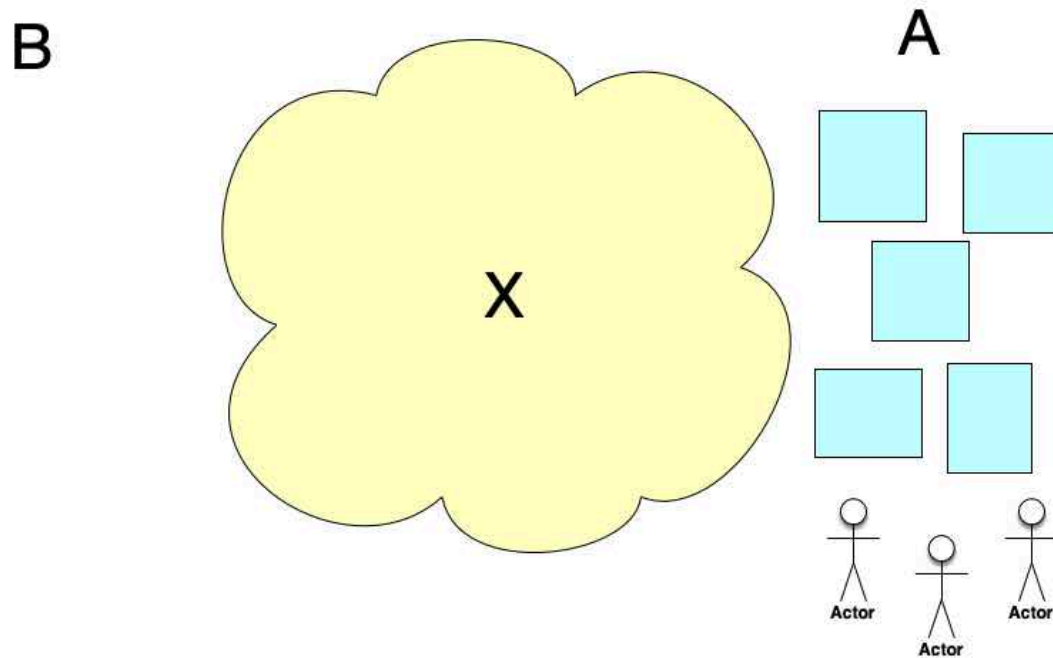
Keep this line in mind; every slide connects to it.

The Game We Play

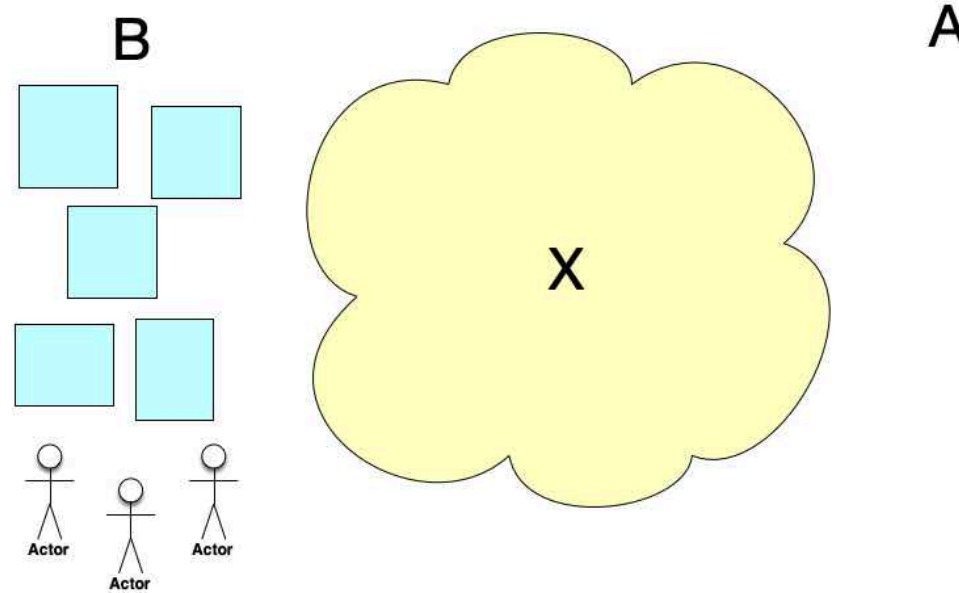
- What's **the name** of the game we are playing?
 - What words express the nature of software engineering?
- What makes something high-quality, especially the software we use every day?



- Software Engineering is basically about moving M from A to B through X.
- This is the game we play.
 - Guess what A, B, M, and X are.

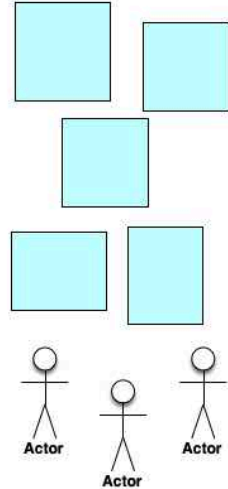


- M is **artifacts**.
 - They are code, tests, docs, and anything that makes the software usable.
- We move M in a **team**.

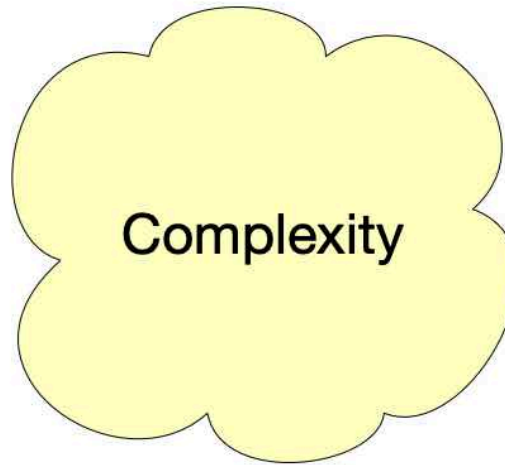


- When we move M from A to B, our job is done.
- The problem is that there is X in between.
- X looks solid, something like concrete, so it seems easy to cross over.

Delivery

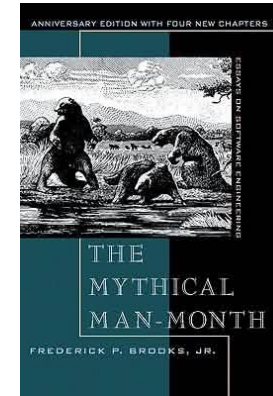


Plan

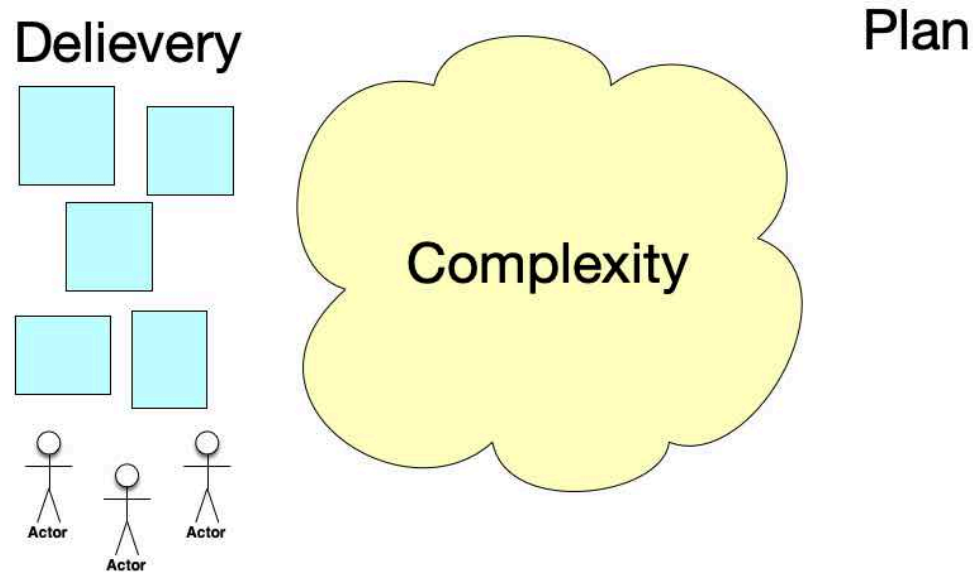


- Unfortunately, no! It is not.
- It is very hard to cross over, because it's a **tar** **pit**, not **concrete**.
- The deeper you are in, the harder it is to get out.

The tar pit metaphor comes from Fred Brooks, "The Mythical Man-Month" (1975).



This is one of the most important software engineering books ever written.



- A is the **plan**.
- B is the **delivery**.
- So, software engineering is about **planning** to **deliver artifacts**; it sounds simple, until we remember the enemy: **complexity**.

It is easy to forget the danger of **complexity**.

- Fred Brooks ("No Silver Bullet", 1986) argued that software may be more complex for its size than any other human construct.
- Complexity is not an accident of bad programmers; it is an **essential** property of software.
- Many projects fail not because people can't code, but because they underestimate this enemy.

What's the name of the game we play (in software engineering)?

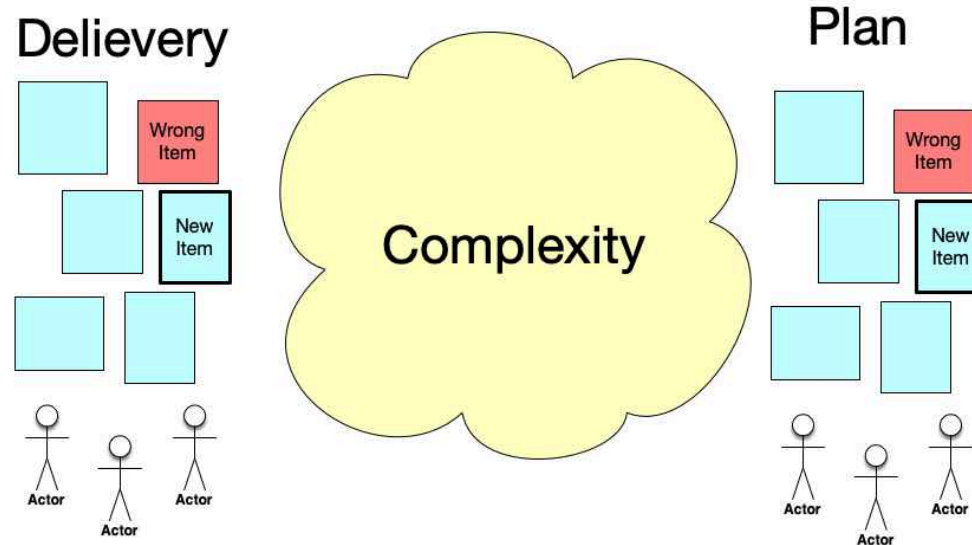
- It is not coding.
- It is not programming.
- It is not even building artifacts.

It is about **managing complexity**.

- The level of complexity is so high that nobody can manage it without proper **rules** and **tools**.
- We should admit that we cannot conquer complexity once and for all; we manage it continuously.

Then, what's the meaning of `high-quality` in this context?

- It's not only about a beautiful UI.
- It's not only about good documentation.



Easy to Add Features
Easy to Fix Bugs

Software Quality Has Many Attributes

The ISO/IEC 25010 quality model lists attributes such as:

- **Functional suitability** - does it do what users need?
- **Reliability** - does it keep working?
- **Performance efficiency** - is it fast enough?
- **Usability** & accessibility - can people use it?

- **Security** - is it safe from misuse?
- **Compatibility** & **Portability** - does it work with others and elsewhere?
- **Maintainability** - can we change it safely?

We cannot study all of these in this course.

| Our Focus Today: Maintainability

Complexity attacks one attribute first: how easy it is to `fix bugs` & `add features`.

- Complexity makes it hard, sometimes impossible, to fix bugs & add features.
- It is common to introduce new bugs when fixing bugs.
- It becomes common sense to expect bugs when adding features.

And when maintainability collapses, the other attributes (reliability, security, ...) usually follow.

So, in this short course, we can use

maintainability to explain high-quality software (HQS).

So, if the software is designed & built in a way that a bug is easily found and easily fixed,

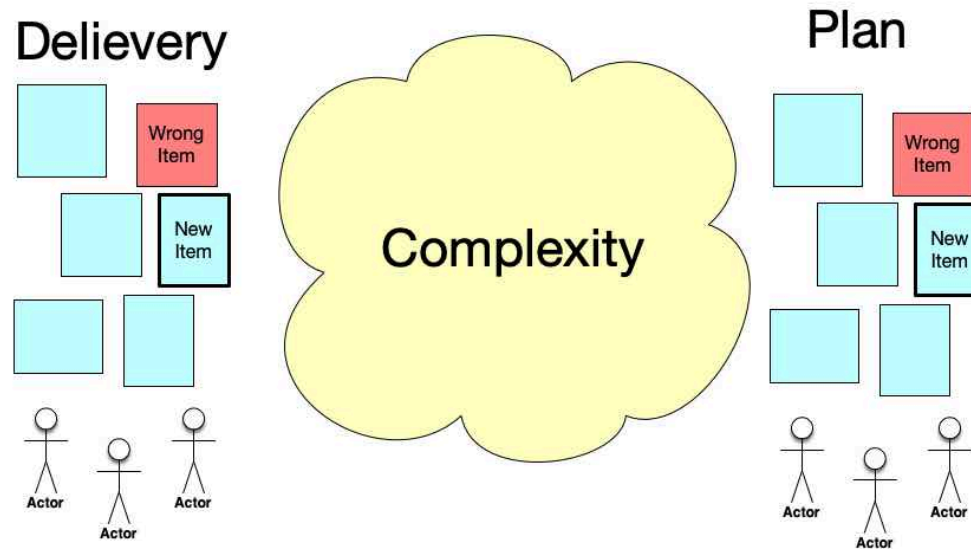
- It is **maintainable** software.

If the software is easy to extend with features that clients request,

- It is **modifiable** software).

These are necessary for high-quality software, but not sufficient.

The HQS is about (1) **fixing bugs** and (2) **adding features**.



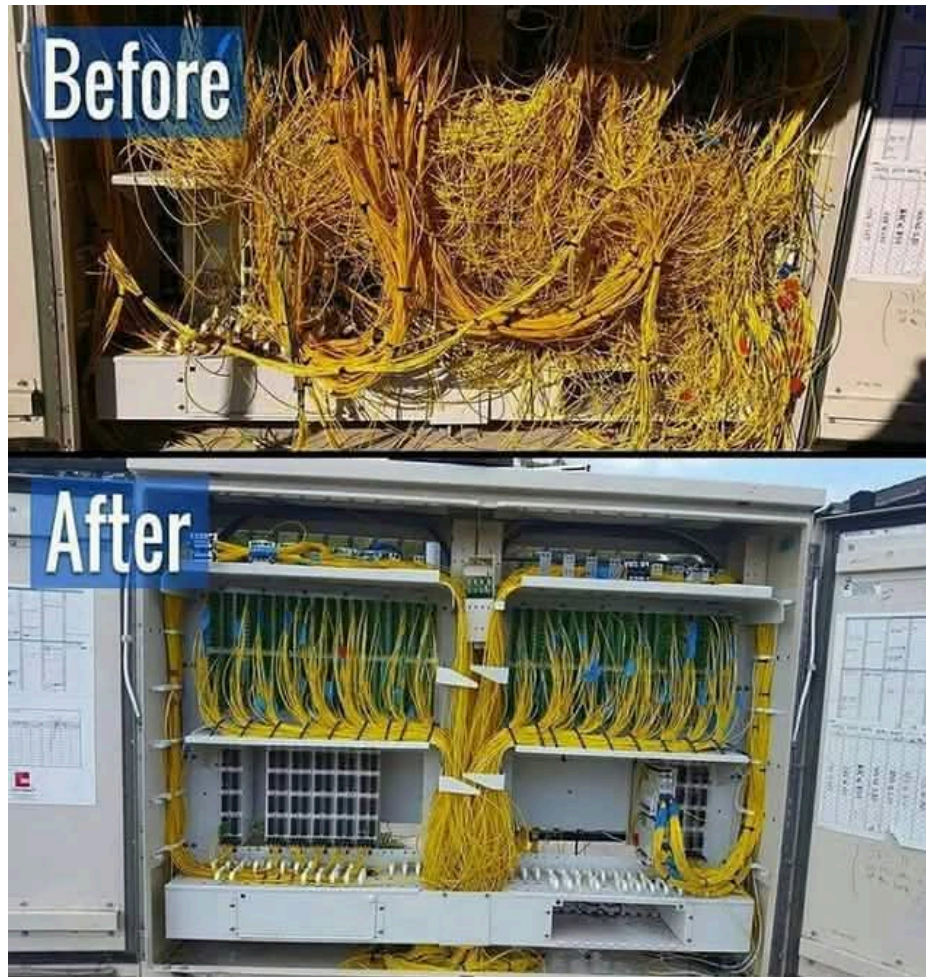
Easy to Add Features
Easy to Fix Bugs

However, it is hard to build **high-quality**
software because of ...

Complexity!

- We need special rules to manage complexity.
- We need special tools to manage complexity.

Otherwise, we can't fix bugs & add features, and users move to better alternatives when they can.



We need to manage complexity to fix bugs and add features.

Other Factors to Consider

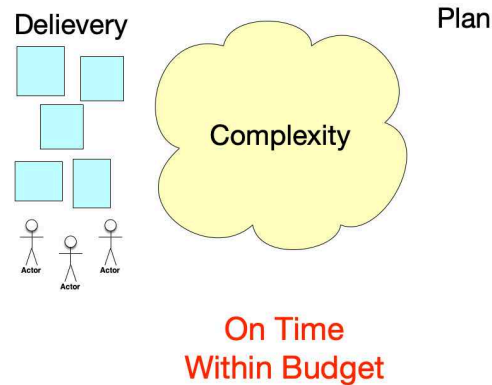
1. Money
2. Time
3. People

| Money

- The artifact is a product in software engineering.
- It's a product that clients **pay** to use.
- Users sometimes tolerate low quality (low price, urgency, no alternatives, switching costs).
- But low quality erodes value over time; when a **better alternative** (competition) appears, users leave.

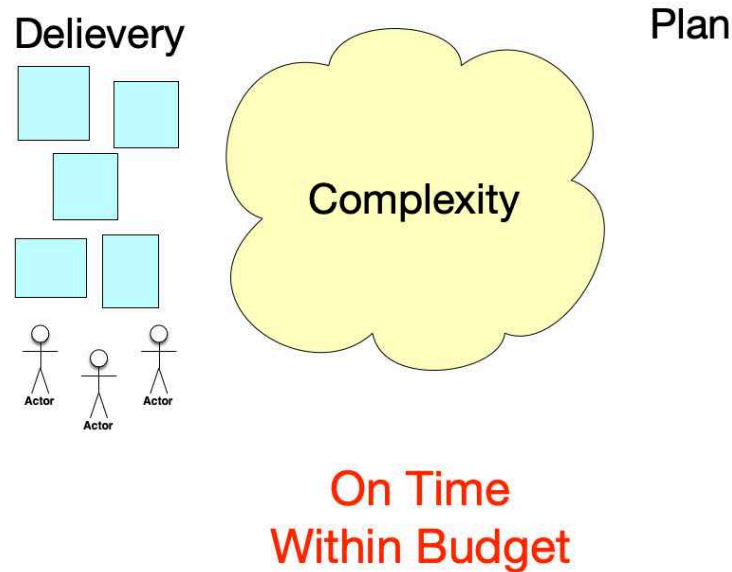
So, the focus is building **High-Quality Software Product (HQSP)**, not just HQS.

Time



- We need to **deliver** the **products** according to the **plan**.
 - On time (missing the deadline costs money)
 - Within budget

People



- We deliver the product in a team.
- Working in a team helps manage complexity, but it is also a new source of complexity (communication, coordination).

As software engineers, we need to solve problems on **time**, within **budget**, and in a **team**.

- Just solving the problem is not enough in a professional setting.
- We should solve the problem within the **constraints**.

Software Engineers

- We know high-quality software products (HQSP):
 - Software that meets users' needs across quality attributes.
 - Especially, software that is easy to fix bugs in and add features to.

- We know software engineering (SWE):
 - Managing complexity to deliver high-quality software products under constraints.

Then, who are software engineers?

- Coders?
- Programmers?

| They are **problem solvers**.

- To deliver high-quality software products on time within budget, they have to solve many problems.
- The problems include:
 - Technology-related problems
 - Business-related problems
 - Human-related problems
 - Many more ...

In software engineering, everyone in the business solves problems:

- CEOs keep the company afloat.
- Architects find the most cost-effective way to build the HQSP.
- Senior software engineers come up with software designs that manage complexity.
- Managers and junior software engineers solve their own problems.

When we solve problems, the hardest part is not

- Technology
- Business
- Human relationships

But ...

How many times have you said

"Oops, I didn't know that."

| The Unknown Unknowns

In many cases, we don't know what we don't know (unknown unknowns).

- Unknown unknowns make software complexity even harder to manage.
- The `plan` and `schedule` need to be updated because of them.

It takes time and effort to learn what we didn't know.
Once we know it:

- Things become much more straightforward.
 - All we have to do is find the solutions.
- We gain confidence.
 - We can more reliably ship the HQSP on time within budget.

| Important Trick in Problem Solving

1. We should identify the **unknown unknowns** as early as possible.
2. Many unknown unknowns are revealed only when we start doing.

So, **make** something early, instead of only reading and thinking.

But research, domain analysis, and threat modeling are also "doing"; sometimes they expose risks faster than code.

- What is important is to reduce overthinking.
 - Unknown unknowns do not reveal themselves just by thinking.
- What is truly important is to start making it.
 - Unknown unknowns reveal themselves when we **make** something.

The Prototype

- A good early step is often a **prototype**.
 - No beautiful architecture or design.
 - No polished tests.
- In making the **prototype**, we begin to see **unknown unknowns**.

Limits: a prototype cannot reveal everything.

- Production scale, security, operations, and real user behavior often stay hidden.

The Minimum Viable Product (MVP)

- The next step is often building an MVP.
- An MVP is the smallest usable product that delivers enough value to **test our most important assumptions** with real users.
 - It has the minimum features.
 - Critical parts (e.g., security, data integrity) still need production-level quality.
 - It is not a complete product with full docs.
- Resist the temptation to build more before you learn from the MVP.

Managing Complexity

We have two approaches for managing complexity.

- Rules
- Tools

| **Software Engineering (SWE) Rules**

What is the most important SWE rule?

KISS: "Keep It Simple, Stupid" (or, politely, "Keep It Simple and Straightforward")

- Choose the simplest solution that satisfies the constraints.
- Follow simple rules.
- Use simple tools.

Warning: a solution that *looks* simple can move complexity somewhere else - find the solution that *is* simple.

| **Software Engineering Tools**

What are the simple rules for using SWE tools?

(1) High-level and (2) Automation.

- Prefer **high-level** tools.
- Let machines solve repetitive problems **automatically**, not humans manually.

Building HQSP

In a way, we are phrasing SWE differently:

- Managing complexity to build HQSP

| How to build HQSP?

What are the tools for HQSP?

Of course, **high-level** and **automation** tools, but before that, we have to discuss two foundational pillars:

1. Software **Process** (managing complexity over time)
2. Software **Design** (managing complexity in structure)

| Software Process: A Simplified Story

Real history is messier; this is a simplified story to show how process manages unknown unknowns over time.

- **Code-and-fix** ("Big Bang"): no process at all; just start coding.
- Engineers realized they need a process to manage complexity.

Waterfall (plan-driven) process

- Borrowed from other engineering fields: requirements - design - implementation - testing, step by step.
- Fun fact: Winston Royce's 1970 paper, often cited as the origin of Waterfall, actually **warned** against a single-pass sequential process and recommended iteration and feedback.

Unknown unknowns again!

- A single-pass plan seems to work, but unknown unknowns break it; we often don't know the real problem when we write requirements or design.
- So, it's very hard to ship the planned product on time within budget.

Iterative Processes and Agile

Iterative and incremental ideas (e.g., Boehm's Spiral model, 1986) led to the Agile Manifesto (2001).

- Core ideas: **iterative delivery**, **feedback**, **collaboration**, and **adaptation**.
- Divide the problem into sizes we can digest.
- Build in **vertical slices** to reveal unknown unknowns quickly.

Today, teams use Agile, hybrid, DevOps, and plan-driven processes (e.g., in regulated domains).

| CI/CD: Automation that Enables Agility

CI/CD is not the definition of Agile, but a key **automation** practice that makes it work.

- **Continuous Integration**: frequently merge changes into a shared codebase and automatically build & test them.
- **Continuous Delivery**: keep the system always in a deployable state.
- **Continuous Deployment**: automatically release every passing change to production.

The **process** to build HQSP

- Instead of planning and building everything at once, we build software little by little with feedback.
- In this way, we reduce the risks from unknown unknowns.

| Software Design

- Modules & Interfaces

How to manage complexity in code?

- Break it down into small pieces.
 - We call each piece a `module`.
- Connect the pieces.
 - We call the connection an `interface`.

This is what engineers/architects do in the real world.

- Before building a house (application), they plan on paper (design) by dividing it into modules (rooms) and connecting them with interfaces (doors).

But be careful: interfaces are also where modules **depend** on each other (**coupling**).

- Too many doors between rooms: changing one room affects many others.
- A good interface is small, stable, and hides the details inside the module (**information hiding**, David Parnas, 1972).

This way, we manage complexity when fixing bugs and adding features.

- We can identify which module causes the problem: fix the bug in that module.
- We can add features effectively by updating or adding modules.

The software **design** to build HQSP.

- By **modularizing** and connecting modules with **interfaces**, we can understand the system at a **high level**.
- High-level understanding lets engineers solve problems with abstractions, delegating details to **automation**.

Managing Complexity through Automation

We can build HQSP with **automation** and high-level tools:

- High-level programming languages
- High-level libraries and frameworks
- Automated tests and static analysis

One of the best ways to manage complexity is to automate.

- Let the compiler translate a high-level language into a low-level language.
- Let the compiler handle complicated low-level details.
- Let frameworks provide `abstractions`, `conventions`, `reusable components`, and `code generators` so we write less repetitive code.

| Example 1 - JavaScript vs TypeScript

JavaScript is dynamically typed, so many type errors appear only at runtime in large applications.

- Microsoft wanted to build large JavaScript applications with fewer bugs, so they created TypeScript (2012).
- TypeScript is a typed superset of JavaScript; its compiler **automatically** checks types and emits plain JavaScript.

```
function area(width: number, height: number): number {  
    return width * height;  
}  
area("10", 5); // Error caught at compile time, not in production
```

Example 2 - React vs Manual DOM Updates

Keeping the UI in sync with changing data has always been hard for front-end developers.

- Facebook (Meta) created React to manage this UI complexity.
- React is a **declarative**, **component-based** UI library: you describe what the UI should look like for a given state, and React **automatically** synchronizes and updates the DOM.

Developers are freed from tedious, error-prone manual DOM manipulation.

- React handles local component state automatically.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

| Example 3 - PHP vs Laravel

PHP is a high-level language for dynamic web servers, but raw PHP leaves many repetitive jobs to developers.

- Laravel is a PHP-based framework that provides abstractions and conventions for routing, database access (ORM), authentication, and validation.
- Its CLI (`php artisan make:...`) generates boilerplate, but developers still write the application logic.

Practices Help, but Do Not Guarantee Quality

Process, design, and automation help manage complexity.

- But a team can use Agile, CI/CD, and React and still ship insecure, slow, or unusable software.
- Quality must still be **specified**, **measured**, and **verified** (the rest of this course).

In other words, building HQSP is not just one practice, it's a big problem solving by a group of people using all the known software engineering rules and tools.

Let's talk about this in this course.

Our Answers

Let's go back to the questions.

Compare these short answers with what you wrote at the beginning.

1. **Software Engineering:** managing complexity to deliver software products under constraints (money, time, people).
2. **High-Quality Software:** software that meets users' needs across many quality attributes; maintainability is the one complexity attacks first.
3. **How to build it:** reveal unknowns early, keep things simple, and use process, design, and automation; then verify quality.
4. **Process & design:** process manages

5. **High-level languages** (e.g., TypeScript): let the machine catch and handle low-level details, so humans focus on the problem.
6. **Automated tools** (e.g., React, CI/CD): machines repeat tasks faster and more reliably than humans.
7. **Frameworks** (e.g., Laravel): conventions and reusable components remove repetitive work, so teams share a common structure.

These are qualified answers; we will refine them throughout the course.